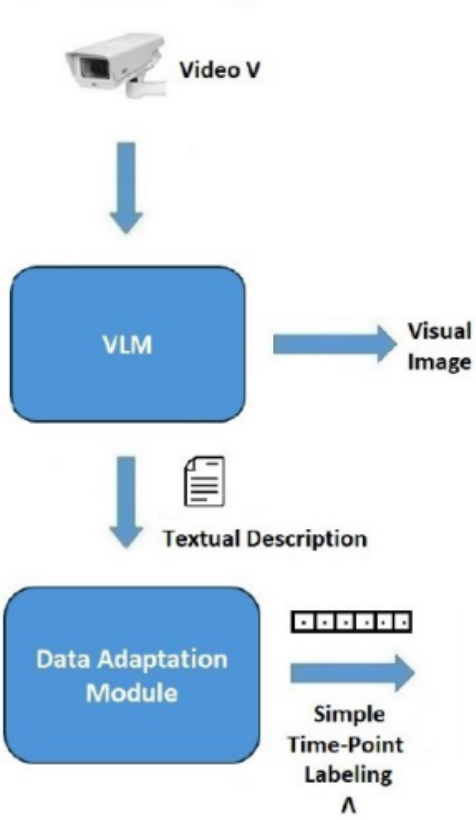
VLM-ISEQL Task Report

1. Task Specification

> In the context of the video surveillance architecture combining VLM and ISEQL, please implement and test the first two levels of the pipeline on video in different contexts (including homemade). As regards VLMs, you could use qwen 2.5 or other versions on ollama, gemini 1.5 robotics view on Google AI studio via API Key, or any other tool deemed appropriate

2 Pipeline Description

As the task requires the implementation of the first two levels of the pipeline, we reported them in the following image, taken from the course slides.



3. Implemented Pipeline

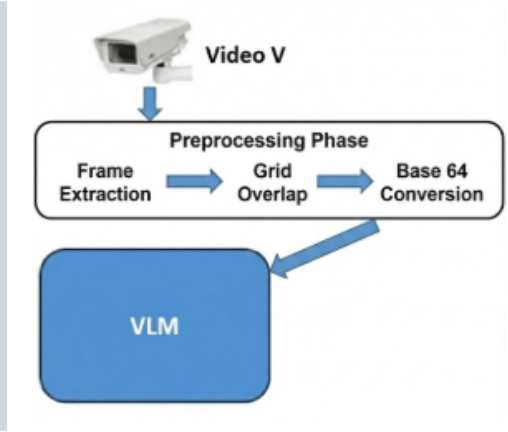
In this section we'll be going through the entire first two-level pipeline implementation, from the video taken in input by the VLM to the Simple Time-Point Label provided to the third level of the entire pipeline.

Obviously, since we are pretty limited from the VLM side, pretty much due to tokens, we'll test this pipeline on simple video and scenarios, like couriers dropping off packages.

3.1 From Video to VLM: Reaching the First Level of the Pipeline

Video recorded by video camera cannot be provided directly to the VLM, there must be a **preprocessing phase**.

First of all frames must be **extracted** from the video and then a grid must be **overlapped**. To provide the VLM with a valid input, we also **convert** frames in **base 64**.



3.1.1 Frame Extraction

Since we're considering simple video and scenarios and don't want to overload the VLM, for the "dropping off packages" topic, extracting 1.5 FPS (1 frame per 0.66 seconds) from a video is sufficient. It's the best trade-off that we could make between VLM image-reasoning completion time and information amount provided per frame.

3.1.2 Grid Overlapping

Referring to the original pipeline, the overlapped grid frame (the so called **Visual Image**) is generated in this phase, before going to the VLM. This is possible through a simple Python script that applies the grid to the frame, without relying too much on the VLM, that is limited to image-reasoning purposes.

Each frame has been structured into a 1x3 matrix, so 1 row and 3 columns, in this case we will refer to them as **3 cells**. This structure will also be used by the VLM to provide descriptions of what happens in each cell of the frame.

As you can see in the following image, no frame label has been used, that is because we obtained better outcomes with this setup from the VLM than with labels.

Frame 160 with grid



3.1.3 Base 64 Conversion

The final step of this first phase in the **base 64 conversion** of each frame, that is the input for the next layer, the VLM.

3.2 From VLM to Data Adaptation Module: From the First to the Second Level of the Pipeline

The first level of the pipeline **analyses frames** in base 64, elaborated in the previous preprocessing phase. Now **Frame-Reasoning** is performed by the VLM to provide the next level a coherent input.

3.2.1 VLM choice

This was by far the most challenging part of the project, since we tried several models locally on ollama with a NVIDIA GeForce RTX 3060 Ti with 8GB of VRAM, but also online even if powerful models comes with limited, very limited tokens online.

So we had to evaluate several aspects, going from online very token-limited models and offline power.

On four video we tested several models with different amount of parameters. Starting offline with our GPU, we tried to test some models:

```
class Models(Enum):
    qwen3_2 = "qwen3-vl:2b"
    qwen3_4 = "qwen3-vl:4b"
    qwen3_8 = "qwen3-vl:8b"
    gemma3_4 = "gemma3:4b"
    gemma3_12 = "gemma3:12b"
    mistral_8 = "ministral-3:8b"
    mistral_14 = "ministral-3:14b"
```

About some of the models listed:

- qwen3_8 is very good in general, but it is very slow on our hardware since it takes ~20 minutes per video analysis
- gemma3_12 is not providing good outcomes... Several analysis are wrong
- mistral_14 is pretty quick since it takes ~15 minutes per frame, but don't recognize packages on the floor

We also tried the online version of gemma3_27b but outcomes were not satisfying at all.

Although not listed here, we would also have liked to try Google AI Robotics 1.5, but it has a RPD and RPM insufficient even for testing, so.. encouraged by several good outcomes offline, we decided to use the offline model `qwen3_8`, since it provides the best trade-off ratio among all the other models.

3.2.2 VLM: Frame-Reasoning

The **base 64** string from the **preprocessing phase** is analysed by the VLM and produces a **textual description** already **serialized**, it is a JSON-string for each cell of the frame.

To achieve such a complex task we tested a lot of prompts and what we noticed is that even with a very-specific prompt, the model was analysing several frames in a bad way.

> **Example**
>
> Real-world context: the courier is dropping off the package on the door's carpet
>
> Deduced context: the courier is going to open the door

This issue was due to the lack of context from the previous frame analysis, in fact this data was not being provided to the VLM to improve further analysis. Providing the VLM such data and running analysis once again, provided good results.

In the following snippet you can find the prompt that was provided to the VLM:

> **Notice that {SCENE_INFO_FEDEX} is the context of the provided video, this row is not included in the prompt**
>
> Prompt:
>
> You are a video surveillance VLM, analyze the image I sent you and reply only with a json representation of what is in each grid cell.
>
> Scene info: {SCENE_INFO_FEDEX}
>
> Each cell in the json is numbered from left to right. If you think that shippers are carrying a box or a package insert it in the json inside objects section. Consider that Boxes or Packages can be placed on the floor by a person or shippers.
>
> Inside the json insert the action of what the person is doing, explain it in the description.
>
> Inside the json action: if the shipper or the person is delivering/carrying the box/package/object insert "carrying an object", if the shipper is releasing the box/package/object insert "releasing an object".
>
> If you don't recognize the objects inside the cells but are related to boxes or packages insert "Other" in the objects section, instead if it is a package insert "package".
>
> If you see boxes or packages not being interacted with anybody don't insert any action, insert the presence of the object in the objects section of the json.
>
> Print the output as a json for all 3 grid cells

And about these cells... Here you can see the structure of each cell in the JSON-string format:

> **Referring to the i-th cell of the frame**
>
> ```
> "cell_i": {
>   "reasoning": "",
>   "people": "",
>   "objects": "",
>   "action": ""
> }
> ```

Since a video is composed by multiple frames, these cells will be packed into a JSON-array string.

This serialized data will be the input for the next level, the Data Adaptation Module.

3.3 From Data Adaptation Module: To the Third Level of the Pipeline

Given serialized data provided by the VLM (the first level of the pipeline), the Data Adaptation Module **deserializes** data and provides input for the next level of the pipeline, the **Interval Action Detector**.

3.3.1 Deserialization and Third Level Input Formatting

This second level of the pipeline is specialized in deserializing the input, retrieving the original JSON structure. This is then **converted** into a suitable format for the next level, respecting the **Single Time-Point Labeling** introduced in the full pipeline.

Several formats could be adopted as an input for the next phase, like the JSON produced in the previous phase or a classical .csv file, also a raw data map can be used, but we decided to implement a **simple relational database** in **MySQL** to address a certain degree of engineerization.

Since our pipeline involves simple scenarios, we still thought about future-proofness, making a easily expandible database with all the necessary relationships.

The database structure is pretty straightforward. We've got our main entities like **Video**, **Person**, **Object**, and **Action** that store the basic info.

Then we have **Video_Frame** which acts as a bridge between videos and their frames - each frame is identified by its frame number and which video it belongs to (using a composite key).

The most important table is **Frame_Cell**, which represents each cell in our gridded frames. This is where all the actual analysis data gets stored, for each cell we save:

- Which frame and video it belongs to
- The cell number (1, 2, or 3 in our case)
- The reasoning text that the VLM generated
- References to what person, object, and action were detected (these are optional foreign keys)

All the foreign key relationships are set up so everything connects properly: frames link to videos, cells link to frames (using that composite key), and cells can optionally reference persons, objects, and actions depending on what was detected in that specific cell.

In this way the DB is ready to continue in the pipeline, for example to extract data with ISEQL.